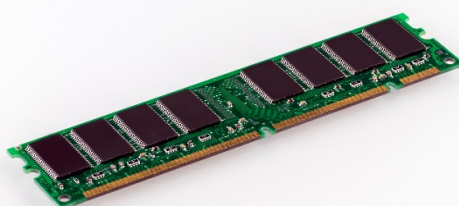


SENSORES, ATUADORES E ARMAZENAMENTO

TEMA 1 – TIPOS DE MEMÓRIA

A memória é um componente eletrônico, muito conhecida nos computadores, mas está presente em quase todo equipamento eletrônico, desde televisão e micro-ondas a algumas cafeteiras. As principais funções da memória são receber, armazenar e informar algum dado. Repare na Figura 1.

Figura 1 – Representação de uma memória RAM



Crédito: Avc Photography/Shutterstock.

A Figura 1 é talvez a imagem mais conhecida de uma memória utilizada em computadores. Antigamente a memória dos computadores usava a unidade de medida kb (kilobytes), depois passou a utilizar mb (megabyte) e atualmente utiliza gb (gigabyte). Como o foco não são as memórias para os computadores, as medidas utilizadas para o armazenamento são um pouco menores. Repare na Tabela 1.

Tabela 1 – Medidas de armazenamentos

Múltiplo	Capacidade (posições de armazenamento)
1 Byte	1
1 Kilobyte	$2^{10} = 1.024 \approx 1 \text{ mil}$
1 Megabyte	$2^{20} = 1.048.576 \approx 1 \text{ milhão}$
1 Gigabyte	$2^{30} = 1.073.741.824 \approx 1 \text{ bilhão}$
1 Terabyte	$2^{40} = 1.099.511.627.776 \approx 1 \text{ trilhão}$

Fonte: INTADM, 2014.

Observando a Tabela 1, pode-se notar que alguns nomes (Gigabyte e Terabyte) nos fazem lembrar de um HD (Hard Disk). O HD utiliza a mesma medida para o armazenamento, e os primeiros HDs estavam na casa dos Bytes. A

unidade mais básica que um eletrônico gerencia é um *bit* (que não está na Tabela 1) e pode receber apenas os valores 0 ou 1. A união de 8 bits é denominada *byte*.

Existem vários tipos de memória, mas para esse material vamos destacar a memória RAM, ROM e as memórias que muitos autores chamam de *memórias híbridas*.

A memória RAM (Random Access Memory) guarda os dados enquanto ela for alimentada eletricamente, apagando os dados quando sua fonte de energia for desligada. Ela também é considerada a memória primária do computador, tendo acesso livre para escrever e ler qualquer dado em qualquer momento.

A memória ROM (Read Only Memory), como o próprio nome diz, é uma memória que não permite gravação, sendo memória somente de leitura. Mas a ROM mantém os dados armazenados mesmo após a interrupção da alimentação de energia. Os principais tipos são: PROM (Programmable Read-Only Memory) e EPROM (Erasable-and-Programmable Read-Only Memory). A PROM permite apenas uma gravação, que depois de concluída só poderá ser acessada (leitura). A EPROM permite que o usuário programe, apague e re programe. Para isso, o usuário utiliza um componente ultravioleta e todos os dados são apagados.

As memórias classificadas como híbridas estão nessa categoria pois podem ser escritas e reescritas como as memórias RAM e ainda armazenam os dados mesmo após a interrupção da alimentação, como as memórias ROM. Sobre as híbridas pode-se citar:

- **EEPROM:** do inglês Electrically-Erasable-Programmable, com as características do EPROM, com a diferença que são programadas eletricamente, e as do tipo EPROM precisam de exposição à luz (raios violetas). A sua gravação é byte a byte, podendo ser uma desvantagem pela velocidade.
- **FLASH:** é o tipo de memória mais utilizada na gravação da programação dos microcontroladores, pela sua velocidade de armazenamento. Elas são escritas eletricamente (igual a EPROM), mas em relação às outras memórias ela é mais rápida, pois agrupam os bytes gerando setores. A gravação é realizada por setor (*Laboratório de Garagem, 2012*).
- **NVRAM:** do inglês Non-Volatile RAM, apresenta uma bateria de backup que entra em funcionamento quando a fonte de alimentação principal é desligada, mantendo a memória sempre alimentada. Um exemplo de utilização é a pilha do computador que mantém algumas informações

armazenadas, mesmo sem a fonte principal (*Laboratório de Garagem*, 2012).

Para finalizar, a memória RAM também tem sua classificação em 3 tipos (SRAM, DRAM, MRAM). O importante para esse material é a SRAM, pois é utilizada no Arduino e retém os dados enquanto é alimentada, apagando quando a alimentação é interrompida.

TEMA 2 – MEMÓRIA ARDUINO

A memória interna do Arduino pode ser considerada reduzida, ainda mais nos dias atuais, em que estamos acostumados a falar em Gigabyte e Terabyte. A capacidade de armazenamento do Arduino UNO é de 32k de memória Flash, 2k de memória SRAM e 1k de EEPROM. Observe a Tabela 2.

Tabela 2 – Capacidade Arduino UNO

Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)

Fonte: Arduino, 2019.

A Tabela 2 é um recorte de uma tabela de informações técnicas do site oficial do Arduino. Como é possível observar, o Arduino apresenta 3 tipos de memória:

- **FLASH (Memória de programa):** É uma memória que não é volátil e que usa a mesma tecnologia dos cartões SD e as memórias USB. Geralmente tem um tamanho entre 32Kb (Arduino UNO) e 512 Kb (Arduino DUE). Nesta memória do Arduino, guarda-se o sketch já compilado, com vida útil de uns 100.000 ciclos de escritura, aproximadamente.
- **SRAM (Static Random Access Memory):** É uma memória volátil, na qual o sketch cria, armazena e manipula variáveis em tempo de execução. Quando interrompemos a alimentação do Arduino ou o desligamos, os dados que estão armazenados simplesmente desaparecem.
- **EEPROM (Electrically Erasable Programmable Read-Only Memory):** Uma memória não volátil e algo mais lenta que a SRAM que geralmente se usa para armazenar dados de configuração ou calibração para que possam sobreviver a um reset, a uma reprogramação

(atualização) do sketch e a um corte na alimentação. Pode-se aceder a esta memória (ler/escrever) fazendo chamadas a partir do próprio sketch.

Para ficar mais fácil o entendimento e a utilização dos três tipos de memória pelo Arduino, observe a Tabela 3.

Tabela 3 – Função das memórias do Arduino

Flash	Armazena o sketch	Não volátil
SRAM	Sketch cria e manipula as variáveis	Volátil
EEPROM	Usados para armazenar informações	Não volátil

O Arduino UNO apresenta apenas 2kb de memória SRAM, e é muito fácil encher esse espaço se o sketch utiliza muitas strings. Observe o exemplo:

```
char texto[ ] = "Eu sou uma string e ocupo muito espaço. ";
```

A variável *texto* coloca 40 bytes na memória SRAM, pois cada caractere utiliza 1 byte (Tabela 4). Essa conta pode não parecer muito, mas é importante levar em consideração dois fatores: primeiro que *texto* é apenas uma variável do sketch (que pode possuir muitas outras), e segundo que o Arduino UNO tem 2kb de espaço, ou seja, apenas 2048 bytes.

Agora que conhecemos os tipos e tamanhos da memória do Arduino UNO (utilizado nesse material), é preciso entender o quanto determinado tipo de dado impacta no armazenamento. Repare na Tabela 4.

Tabela 4 – Tipos de dados e seus tamanhos de armazenamento

Tipo de dado	Tamanho em Bytes
char	1
boolean	1
int	2
long	4
float, double	4

Há ainda a possibilidade de utilização do comando *unsigned* nos tipos de dados int e long, não permitindo assim a utilização de números negativos. Repare na Tabela 5.

Tabela 5 – Utilização do unsigned

int	-32768 até 32767
unsigned int	0 até 65535
long	-2147483648 até 2147483647
unsigned long	0 até 4294967295

O importante é que int ou unsigned int utilizam o mesmo tamanho de armazenamento demonstrado na Tabela 2. A mesma coisa acontece com o long.

Só por questão de curiosidade, o site oficial do Arduino trás as informações técnicas de todos os tipos de Arduinos. O Arduino Mega 2560, também muito utilizado, é um pouco superior ao UNO, sendo utilizado em projetos maiores. Repare na Tabela 6, que traz as informações das memórias do Mega 2560.

Tabela 6 – Capacidade Arduino Mega 2560

Flash Memory	256 KB of which 8 KB used by bootloader
SRAM	8 KB
EEPROM	4 KB

Fonte: Arduino, 2019.

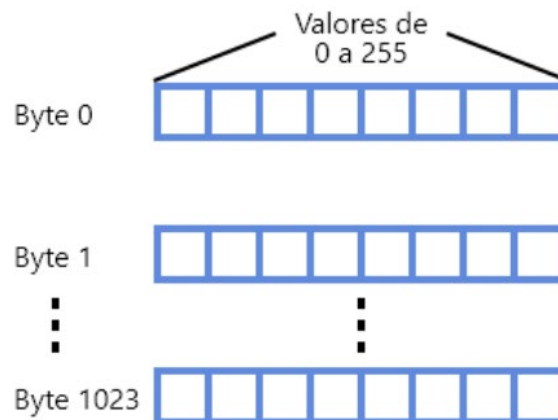
TEMA 3 – ENTENDENDO A MEMÓRIA EEPROM

A memória EEPROM pode ser útil quando for necessário armazenar dados que não podem ser perdidos; mesmo se a alimentação do Arduino for interrompida (memória não volátil), os dados ficam lá intactos. Quando o Arduino for alimentado novamente, os dados podem ser acessados. Mas é de extrema importância utilizar essa opção com cautela, pois existem dois fatores importantes:

- Espaço: que no Arduino UNO é de apenas 1k
- Limitação no número de gravações

Exemplificando, para armazenar o estado de uma lâmpada (ligada ou desligada), é preciso apenas 1 bit. Cada byte tem 8 bits. Para cada byte, então, é possível armazenar o estado de 8 lâmpadas. O Arduino UNO tem 1k de memória, ou seja, 1024 bytes, que teoricamente armazena 8192 estados. Lembrando que cada byte pode ser armazenado um valor entre 0 e 255, que em decimal representa 00000000 e 11111111 respectivamente. Observe na Figura 2.

Figura 2 – Representação da estrutura memória EEPROM



Fonte: Fábio, 2018.

Conhecer a estrutura apresentada na Figura 2 é importante, pois na gravação ou leitura é utilizado os endereços de 0 até 1023 para acessar os bytes.

Agora que sabemos, na teoria, para que serve e qual a utilização da memória EEPROM, concentre-se e acompanhe o exemplo prático.

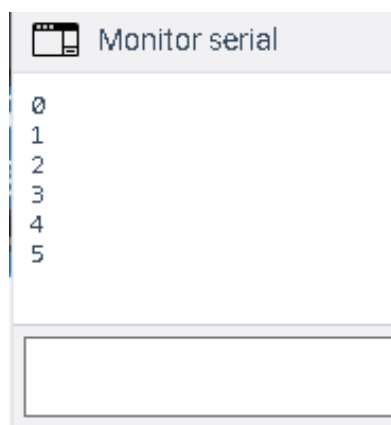
Para esse exemplo será utilizado apenas um Arduino UNO e o cabo USB (pode ser programado no simulador também). O exemplo consiste em um contador simples que é iniciado com o valor 0, que de 1 em 1 segundo imprime o valor incrementado no monitor serial. Observe a Figura 3.

Figura 3 – Exemplo de um contador simples

```
1 // Entendendo a Memória EEPROM
2
3 int contador = 0;
4
5 void setup() {
6   Serial.begin(9600);
7 }
8
9 void loop() {
10  Serial.println(contador);
11  contador++;
12
13  delay(1000);
14 }
```

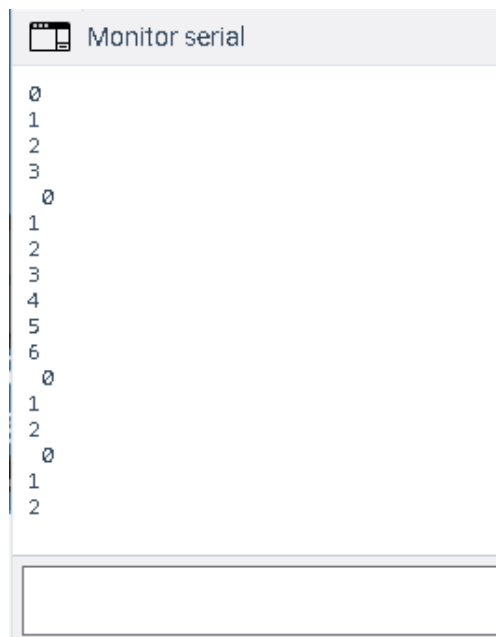
Para testar o programa, execute e verifique no monitor serial que deve apresentar como resultado a Figura 4.

Figura 4 – Representação do resultado do sketch da Figura 3



Na programação do sketch da Figura 3, não há um limitador e a impressão dos números será executada enquanto o Arduino estiver ligado. Observe atentamente a Figura 3 e verifique que **não** foi utilizada a memória EEPROM, apenas a memória flash. Para confirmar, com o Arduino ligado aperte o botão reset que o contador voltará para o número inicial (zero).

Figura 5 – Sketch da Figura 3 sendo reiniciado



Repare na Figura 5 que o botão reset foi pressionado 3 vezes, inicializando novamente o contador. Isso ocorre pois os dados estavam apenas sendo armazenados temporariamente na memória flash (volátil). Se o projeto precisa que esse valor não se apague cada vez que o Arduino é desligado ou reiniciado, é utilizado a memória EEPROM. Acompanhe o sketch na Figura 6.

Figura 6 – Sketch utilizando a memória EEPROM

```
1 // Entendendo a Memória EEPROM
2 #include <EEPROM.h> // biblioteca padrão
3
4 int contador = 0;
5
6 void setup() {
7     Serial.begin(9600);
8
9     if (EEPROM[0] != 9) // verifica se não foi gravado algo
10    {
11        EEPROM[0] = 9;
12        EEPROM[1] = contador; // grava na posição 1
13    } else {
14        contador = EEPROM[1];
15        // move o valor gravado para a variável contador
16    }
17 }
18
19 void loop() {
20     Serial.println(contador);
21     contador++;
22     EEPROM[1] = contador;
23
24     delay(1000);
25 }
```

Na linha 2 foi incluída a biblioteca padrão EEPROM.h, responsável pelas funções e configurações para ser utilizada a memória EEPROM. Dentro do setup é definido a comunicação serial e um teste (linha 9) para saber se possui algo gravado na EEPROM ou não. O valor 9 é utilizado apenas como marcador. Se a EEPROM estiver limpa, o sketch atribui o 9 como marcador na posição 0 e move o valor do contador para a posição 1. Se a memória EEPROM já tiver preenchida, é pego o valor gravado na posição 1 da EEPROM e movido para a variável contador (linha 14).

O loop do sketch é simples e utiliza sempre a variável contador. Primeiro é impresso, no monitor serial, o valor da variável contador, depois ela é incrementada e por último é gravado esse novo valor na posição 1 da EEPROM. Repare na Figura 7.

Figura 7 – Representação do sketch da Figura 6



Observe a Figura 7 que, quando o contador estava em 9 o Arduino foi resetado, mas a contagem continuou normalmente. Para finalizar o tema, é importante lembrar que na posição EEPROM[1], que foi utilizada na Figura 6, é possível armazenar apenas até o número 255.

TEMA 4 – ARMAZENANDO DADOS (EEPROM)

A biblioteca EEPROM.h é uma biblioteca padrão que possui as funções necessárias para a utilização da memória EEPROM. A leitura dos dados armazenados em um EEPROM é praticamente infinita, mas a sua reprogramação, ou seja, apagar os dados e programar novamente tem uma vida útil limitada. Isso ocorre porque o chip sofre uma pequena deterioração cada vez que o conteúdo é apagado, pois a exclusão requer uma tensão elétrica mais elevada. Mas fiquem tranquilos, pois a sua vida útil, mesmo limitada, passa das centenas de milhares de vezes. Para facilitar a compreensão, os comandos foram separados por categorias.

Existem três funções que podem ser utilizadas para fazer a gravação de valores na memória EEPROM:

EEPROM.write(endereço, valor_byte)

A função write possui 2 parâmetros: endereço começa com seu valor em 0 e vai até a capacidade da memória (valores entre 0 e 1023 no Arduino UNO) e

valor_byte (valores entre 0 e 255). Comando bem simples e direto, que grava exatamente o valor no endereço informado.

Figura 8 – Representação da função write

```
1  #include <EEPROM.h>
2
3  void setup()
4  {
5      //preenche as 255 posições iniciais com zero
6      for (int i = 0; i < 255; i++)
7          EEPROM.write(i, 0);
8  }
```

Fonte: Elaborado com base em Tavares, 2017

EEPROM.update(endereço, valor_byte). Essa função é semelhante à função write, mas o valor só é gravado se o valor já existente na memória for diferente do novo valor. Essa função ajuda a aumentar a vida útil da EEPROM, pois evita que dados repetidos sejam gravados.

Figura 9 – Representação da função write

```
1  #include <EEPROM.h>
2
3  void setup()
4  {
5      //preenche as 255 posições iniciais com zero
6      for (int i = 0; i < 255; i++)
7          EEPROM.update(i, 0);
8      // se a posição estiver com zero, não grava
9  }
```

Fonte: Elaborado com base em Tavares, 2017

EEPROM.put(endereço, dado). Exatamente a mesma função do update, diferenciando apenas no tipo de dado gravado. Os dados gravados podem ser do tipo primitivo ou estruturas. Explicando melhor, a função EEPROM.put não aceita apenas valores entre 0 e 255.

Figura 10 – Representação da função put

```
1  #include <EEPROM.h>
2
3  struct stdataInicio
4  {
5      byte bDia;
6      byte bmes;
7      int  nAno;
8  };
9
10 void setup() {
11     struct stDataInicio dataInicio;    // variável que será escrita
12
13     dataInicio.bDia= 02;
14     dataInicio.bMes= 10;
15     dataInicio.nAno= 2017;
16
17     //grava a partir do endereço 10 os dados em dataInicio
18     EEPROM.put(10,dataInicio);
19 }
```

Fonte: Elaborado com base em Tavares, 2017

Para realizar a leitura ou extração de dados já gravados em uma EEPROM existem 2 funções: EEPROM.read(endereço). A função read retorna o valor que está armazenado no endereço que foi passado pelo parâmetro.

Figura 11 – Representação da função read

```
1  #include <EEPROM.h>
2
3  void setup()
4  {
5      byte value= EEPROM.read(0); //le o byte na posição zero
6  }
```

Fonte: Elaborado com base em Tavares, 2017

EEPROM.get(endereço, tipo_dado). Essa função também é utilizada para retornar um valor armazenado na memória, mas além do endereço é passado o tipo de dado (int, float, char) como parâmetro. A função get não retorna apenas o endereço passado no parâmetro, dependendo do tipo de dado ela retorna os valores seguintes até todo o dado estar completo. Exemplificando, foi gravado um dado do tipo float, com o comando put, na posição 10. O float, como já visto, ocupa um espaço de 4 bytes (10, 11, 12 e 13). Assim, ao utilizar o get, o retorno vai ser das 4 posições, não apenas da posição 10 (inicial).

Figura 12 – Representação da função get

```
1 #include <EEPROM.h>
2
3 void setup(){
4     float fVar = 0.00; // variável que será lida
5
6     //le a variável float iniciando na posição zero
7     EEPROM.get( 0, fVar );
8 }
9
```

Fonte: Elaborado com base em Tavares, 2017.

Segundo o site do Arduino, uma operação de leitura ou escrita na memória EEPROM demora 3,3ms para ser efetivada, o que é uma velocidade muito baixa, caracterizando as memórias EEPROM como lentas.

TEMA 5 – ARMAZENANDO DADOS – CARTÃO SD

O Arduino, ou qualquer outro tipo de microcontrolador, independentemente do modelo, sempre vai ter uma limitação de memória. Em alguns projetos, é preciso ter um recurso de memória maior, sendo necessário expandir essa memória de armazenamento, e o micro SD é uma das soluções propostas. Observe a Figura 13.

Figura 13 – Representação de um micro SD

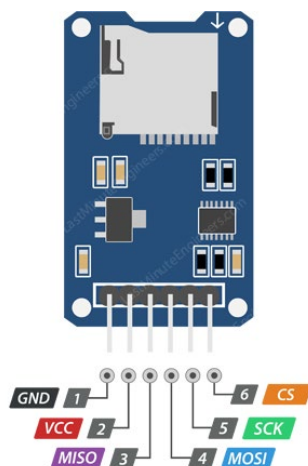


Crédito: Toozdesign/Shutterstock.

Os micros SDs são utilizados em diversos aparelhos eletrônicos com a função de ampliar a memória utilizada para o armazenamento. Sua utilização é comum em aparelhos celulares e câmeras fotográficas. O Arduino utiliza o SD para a mesma função, ampliar a sua capacidade de armazenamento, e para isso

é necessária a utilização de um módulo de micro SD card. Repare na sua representação na Figura 14.

Figura 14 – Representação de um módulo micro SD card



Fonte: Elaborado com base em *Last Minute Engineers*, 2019.

O módulo micro SD card é utilizado para conectar o SD card a um Arduino. Essa conexão pode ou não utilizar uma protoboard. O Arduino trabalha com 5v e o módulo micro SD trabalha com 3.3v, mas não se preocupe, pois o módulo possui um regulador de tensão e um alternador de nível lógico, não sendo necessário montar um circuito divisor de tensão. Note na Figura 14 que o módulo possui 6 pinos. O GND já é conhecido e o VCC é a alimentação. Os outros 4 pinos são conhecidos como “comunicação SPI” que é um barramento mais complexo, mas também mais rápido, ideal para o SD card.

Antes de começar a explicar os pinos do módulo micro SD, é importante destacar os três métodos mais conhecidos de comunicação serial: UART, SPI e I2C. Observe a Tabela 7.

Tabela 7 – Tipos de comunicação serial

Tecnologia	Barramento de comunicação	Taxa máxima	Fluxo de dados
UART (RS232)	2 (sem controle de fluxo)	115.200 bps	Half ou Full Duplex
SPI	3 + nº de Slaves	2 Mbps	Full Duplex
I2C	2 (até 127 dispositivos)	400 Kbps	Half Duplex

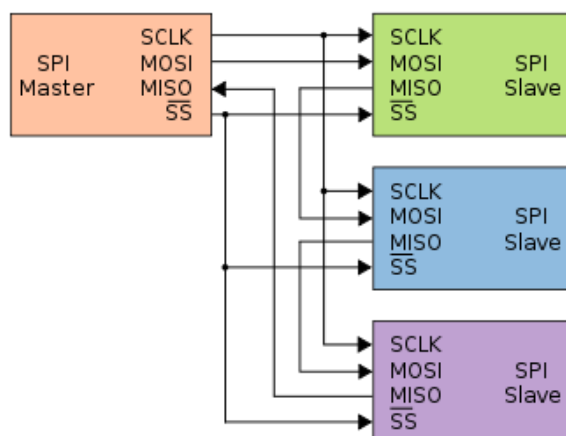
Fonte: Sacco, 2014.

Os dispositivos SPI trabalham em “full duplex” (transmite e recebe dados ao mesmo tempo) e não há limites de dispositivos escravos plugados ao

dispositivo mestre. O site *Robocore* (2019) explica o funcionamento: “Primeiramente o mestre gera um clock e seleciona através do pino SS com qual dispositivo será efetuada a comunicação. Em seguida os dados são enviados para o dispositivo de destino pelo pino MOSI e então o dispositivo escravo envia uma resposta (se necessário) ao mestre pelo pino MISO”.

Repare na Figura 15, que representa a estrutura mestre-escravos.

Figura 15 – Representação da estrutura mestre-escravos



Fonte: Wikipédia, 2019.

Pode-se notar, na Figura 15 que a taxa de transferência do SPI é a maior entre elas. Mesmo os fabricantes tendo a liberdade de desenvolver seus dispositivos com diferentes taxas, os que utilizam o SPI sempre possuem a melhor opção. Verifique na Tabela 8 as funções de cada pino.

Tabela 8 – Funções dos pinos do SPI

Nome	Pino do Arduino	Descrição
MOSI	Elegível. Sugerido: 11	Master Output Slave Input, ou seja, saída do master para a entrada do slave.
MISO	Elegível. Sugerido: 12	Master Input Slave Output, ou seja, a saída do slave vai para a entrada do master.
SCK	Dependente do Arduino. No Uno o pino é o 13	SCK é Serial Clock. Cada Arduino possui o barramento SPI em um lugar diferente e será necessário consultar o pinout conforme a board utilizada.
CS	Dependente do Arduino. No Uno o pino é o 4	CS significa Chip Select. Em alguns casos você pode encontrar a descrição SS, que significa Slave Select.

Fonte: Elaborado com base em *Do bit ao byte*, 2018.

REFERÊNCIAS

ARDUINO. **Arduino Uno review**. Disponível em: <<https://store.arduino.cc/usa/arduino-uno-rev3>>. Acesso em: 3 jun. 2019.

COMPARAÇÃO entre protocolos de comunicação serial. **Robocore**. Disponível em: <<https://www.robocore.net/tutoriais/comparacao-entre-protocolos-de-comunicacao-serial.html>>. Acesso em: 3 jun. 2019.

FÁBIO. Gravar dados na EEPROM do Arduino – Aula 13 – AI. **Mundo Projetado**, 31 out. 2018. Disponível em: <<http://mundoprojetado.com.br/gravar-dados-na-eprom-do-arduino/>>. Acesso em: 3 jun. 2019.

INTADM. Os Componentes de um Sistema de Computador. **Infoblog**, 7 maio 2014. Disponível em: <<https://infotecadm.wordpress.com/2014/05/07/os-componentes-de-um-sistema-de-computador/>>. Acesso em: 3 jun. 2019.

INTERFACING micro SD Card Module with Arduino. **Last Minute Engineers**. Disponível em: <<https://lastminuteengineers.com/arduino-micro-sd-card-module-tutorial/>>. Acesso em: 3 jun. 2019.

LER e escrever no SD card com Arduino. **Do Bit ao Byte**, 2018. Disponível em: <<https://www.dobitaobyte.com.br/ler-e-escrever-no-sd-card-com-arduino/>>. Acesso em: 3 jun. 2019.

OS TIPOS de memórias encontrados em microcontroladores. **Laboratório de Garagem**, 18 jun. 2012. Disponível em: <<http://labdegaragem.com/profiles/blogs/artigo-os-tipos-de-mem-ria-encontrados-em-microcontroladores>>. Acesso em: 3 jun. 2019.

SACCO, F. Comunicação SPI – Parte 1. **Embarcados**, 2014. Disponível em: <<https://www.embarcados.com.br/spi-parte-1/>>. Acesso em: 3 jun. 2019.

SERIAL Peripheral Interface. In: **Wikipedia**. Disponível em: <https://en.wikipedia.org/wiki/Serial_Peripheral_Interface>. Acesso em: 3 jun. 2019.

TAVARES. A EEPROM interna do Arduino. **Caderno de laboratório**, 4 out. 2017. Disponível em: <<https://cadernodelaboratorio.com.br/2017/10/04/eprom-interna-do-arduino/>>. Acesso em: 3 jun. 2019.